上板执行核对单（达芬奇-lite + AD9238）

> 本文把 `docs/06_board_test_guide.md` 收敛成"上板当天照做"的一页核对单。 **每个数字都标了出处**（见 `file:line`）；没有出处的数字一律不写。 顺序：★ 一页速查表（文首）→ 0 前置 → 1 烧 ILA 版抓第一帧 → 2 烧生产版 + UART → 3 输出与信号完整性 → 4 已知坑 → 5 速查表（已上移）→ 6 预算与回退。

★ 一页速查表（上板当天只看这一页；每一步的数字判据与失败回退都在表里）

步骤	命令 / 操作	判据	回退
1 接线	载板插 `J6`； `J10` / `J9` →3.3 V；信号源 10 Vpp/0 V/90°/1 Hz 经同轴到 `J1` / `J2`；线 ≤5 cm	万用表： `J1` / `J2` 对地 ≈4 kΩ、 `AD1CLK` 与 `AD2CLK` 不相通	插到 J7 → 改插 J6；跳线不对 → 拨 3.3 V
2 抓 ADC	**先把信号源改到 100 kHz / 10 Vpp / 0 V**（1 Hz 时窗口内基本是常数，见文末的频率警示）； Program Device → `_synth_work/enc_top_ila.bit` + 同目录 `enc_top_ila.ltx`（大小/时间戳见 §1.1）	**36 条命名探针 / 269 bit**（v12 现行： `dbg_s_raw_11_0_` = probe26、 `dbg_c_raw_11_0_` = probe9；2026-10-10 16:12 轮为 34 条 / 252 bit、probe24/probe7）； **`dbg_s_raw_11_0_` + `dbg_c_raw_11_0_` 中心 `0x000`（二进制补码，不是 偏移二进制）、相差 90°、非全 0**	只有 `probe0[xx]` → 加载 `.ltx`； 下载卡死 → JTAG ≤10 MHz；读数像位序错乱 → 见 §1.6 的位序警告
3 时钟	示波器 `J6-26` / `J6-9`	两路 **50 MHz 同相**、3.3 V 摆幅	无波形 → 查 `J10`、重下 bit；反相 → 查两路生成时钟约束
4 相位余量	双通道探 **CH1 = `ad_clk`（`J6-26`）** 与 **CH2 = `CH1_D11`（`J6-38`，符号位)**，量 `tPD_meas`。**★ 不要探 `D0`**： 满幅正弦下 D0 每秒跳变数百万次，5 ns/div 只有 50 ns 屏宽 ⇒ 必然"糊成一片"（不是故障）； `CH1_D11` 只在过零点翻转（1 kHz 输入 ⇒ 20.18 kHz），触发挂 CH2 上升沿	**≤4 ns** 继续；**≥5 ns 必须改相位**（**2026-10-10 实测通过**：示波器读数 ≤4 ns ⇒ 沿用上升沿捕获， `rtl/ad9238_capture.v:79/:94/:139` **不改**，见 DEV_STATUS §1.27； **（2026-10-12 v11）复测 `tPD_meas = 3–4 ns`** ⇒ 建立余量 ≈ **+12.3 ns**，**回退动作确认不需要**，见 DEV_STATUS §1.27 与 `docs/09_scope_measure_log.md` §1 测量 A）	① 缩线/串 22~33 Ω ② **捕获改 `clk_i` 下降沿**（`rtl/ad9238_capture.v`） ③ 降 25 MHz ④ 最后才 `set_false_path` 并在 DEV_STATUS 记明。**★ （2026-10-12 v11）①②③④ 均确认不需要执行**
5 逐位验证	信号源改 **0.5 Hz 三角波 / 10 Vpp**； `python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --csv chk.csv`	`s_n` 从最负**单调**爬到最正再回落，无台阶/回跳；示波器上 `CH1_Dk` 翻转频率 = `CH1_D0` 的 `1/2^k`	出现台阶 → 该位错线，按 `J6` 针→球号表逐脚查

步骤	命令 / 操作	判据	回退
6 遥测	`python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --csv adc.csv --png adc.png`	`frames>0`、0 校验错；`angle` 单调 0→65535；幅度 ≥19660；`stat=0x00`	先查校验和约定（含帧头）→ 再查波特率 2000000 → 最后查 `0x0109` 方向位
7 ABZ/Z	示波器 `J5-5/6/7`	A/B 正交（1 kHz 下各 ~256 kHz、B 滞后 A 977 ns）；Z **每 64 个输入周期** 1 个（= f_in/64，1 kHz ⇒ 15.625 Hz），宽度 = `0x0108` 个 **AB 边沿步**（★不是 50 MHz 拍：1 kHz/inter=256/div=1 时 1 步 977 ns ⇒ 默认 4 ⇒ 3.9 µs；想看清写 `0x0108=0x000F` ⇒ 14.6 µs）	Z 不动 → 查 `0x0108` 是否为 0；宽度不对 → 见 DEV_STATUS §1.26
8 标定	按 `KEY0`（`W9`）或 SPI 写 `0x0502=0x0001`	`led[3]` 亮一下；`--png` 李萨如图更正圆	标定后更差 → 激励要 1 Hz 慢扫、不可堵转
9 UVW	SPI 写 `0x0109=0xC0`（要好看再加 `0x0102=0x1C`）	**1 kHz 输入、`0x0100=4`（默认）⇒ 每相基波 = 4 × f_in = 4 kHz（周期 250 µs）、50 % 占空、互差 120°（相邻相滞后 83.3 µs）**；"`a_out` 边沿率 ≈ 64 Hz"那条只在**信号源 0.5 Hz + div=8** 的推荐配置下成立（docs/06:454-461）	全低 → `uwven` 没打开（默认 `0x0109=0x40`）
10 收工	存照片（§3.6）+ CSV + `stat` 记录	8 条验收项（docs/06 §8）逐条打勾	未过的项写进报告"已知缺口"，不要粉饰

SPI 常用包（32 位，`1 00 01 <page[2:0]> <addr[7:0]> <data[15:0]>`）：`0x0109=0xC0` 开 UVW；`0x0102=0x1C` 得 `div=8`；`0x0100=0x08` 极对数 8；`0x0108=0x400F` Z 宽 15 个 AB 边沿步（1 kHz/div=1 ⇒ 14.6 µs；★ **必须带高字节 `0x40`** —— 写 `0x000F` 会把下一地址 `0x0109` 写成 0x00，`abzen` 被清、Z 无输出；量完写回 `0x4004`）；`0x0502=0x0001` 一键标定；**`0x0405=0x0001` 清 ADC sticky OTR**（`led[0]`/`irq_n` 被锁存后的唯一软清法，2026-10-10 新增，见 §1.9）（见 docs/06:374-380）。

> ★ **没有 SPI 主机也能发这些包**（2026-10-10 新增，见 §4.13 与 `docs/06` §5.1）： 位流里已集成 **UART→SPI 桥**，把板载 USB 串口插上电脑，用 `python tools\uart_spi.py --port COM7 write 0x0109 0x00C0` 即可； 手工敲就是往串口发 **4 个字节**（2 Mbaud、8N1、HEX）： `0x0109=0xC0` → `89 09 00 C0`； `0x0405=0x0001` → `8C 05 00 01`； `0x0502=0x0001` → `8D 02 00 01`； `0x0102=0x001C` → `89 02 00 1C`； `0x0108=0x0008` → `89 08 00 08`；读 `0x05F0` → `85 F0 00 00`。 第 1 字节 = `0x80 | (wm<<3) | page`（`wm=1` 写 / `0` 读， `page = addr[10:8]`）。

---

0. 上板前置（断电做完再上电）

#	动作	判据	出处
0.1	载板 40 针排母插 `J6`（pin1 对 pin1）；**不要插 `J7`**（本板是 RGB_TFTLCD 液晶口），本板**没有 `J11`**	载板插在 J6、方向正确	见 docs/06:39、docs/06:41-42
0.2	跳线 **`J10`（VCCIO_B16）→ 3.3 V**、**`J9`（VCCIO_B15）→ 3.3 V**（由可调 LDO `U22` 经三位跳线选择）	两个 bank 都在 3.3 V。**出厂不保证 3.3 V**；不对则 DRC 报 "Bank 16 / Bank 15 VCCO 与 IOSTANDARD 不符"，重则损伤 IO	见 docs/04:16、docs/04:256-259、docs/06:48-49、docs/06:51-53、docs/06:513
0.3	载板供电取自开发板 `J6-1`（`+5V`），**约 0.4 A**	分级上电第 1 级：只接 +5V/GND，电流约 0.4 A、载板微温不烫	见 docs/04:108、docs/04:275-278
0.4	信号源：**两路独立单端、共地**；每路 **10 Vpp / 高阻 / 0 V 偏置**，CH2 相对 CH1 **相差 90°**，起始 **1 Hz** 正弦	载板输入范围 ±5 V（`Vout=(1.0/5.02)*vin`），1 LSB ≈ 2.44 mV	见 docs/06:71、docs/06:93-101、docs/06:109
0.5	CH1 芯线→载板 `J1`(CH1)、CH2 芯线→载板 `J2`(CH2)；用 **50 Ω BNC→SMA 同轴线**，**不要用杜邦线接模拟输入**	屏蔽都落在载板地（两路共地），不存在负端短地问题	见 docs/06:73-75、docs/06:81-84、docs/06:87-88
0.6	数据/时钟线 **24 根数据 + 2 根独立时钟（`B18`→`AD1CLK`、`F16`→`AD2CLK`）+ 2 根 OTR**，建议 **≤5 cm**、最好排线	万用表确认 `AD1CLK` 与 `AD2CLK` **在载板上不相通**（相通就停下，不要短接）	见 docs/04:110、docs/04:282、docs/06:136、docs/06:20
0.7	输出侧全部走 `J5`（不在 J6）：`a_out`=J5-5、`b_out`=J5-6、`z_out`=J5-7、`u_out`=J5-8、`v_out`=J5-9、`w_out`=J5-10、`led[2]`=J5-18、`led[3]`=J5-19、`irq_n`=J5-20、`spi_sck`=J5-14、`spi_ss_n`=J5-15、`spi_mosi`=J5-16、`spi_miso`=J5-17	探头能对上球号：`H20/G20/M21/L21/N22/M22`、`J21/K21/K22`、`N20/M20/L19`、`L20/J22/H22/J20`	见 constr/enc_top.xdc:235-240、constr/enc_top.xdc:245-247、constr/enc_top.xdc:254-257、constr/enc_top.xdc:276-277、constr/enc_top.xdc:285
0.8	断电万用表检查：模块针 1(+5V) 与针 2/4/5(GND) 不短路；`J1`/`J2` 芯线对载板地约 **4 kΩ**（4.02K 分压电阻）；信号源接上但未开输出时芯线对地 **0 V**	全部打勾才上电	见 docs/06:113-115、docs/06:134-137
0.9	仪器与软件：示波器（≥2 通道 / ≥100 MHz）、上位机笔记本 + USB 线（板载 FT2232 同时给 JTAG 与串口）、Vivado Hardware Manager、`tools/uart_scope.py`	板载现成指示：`led[0]`=`AB21`、`led[1]`=`AB22`、`key_n`=`W9`、`uart_tx`=`W21`	见 docs/06:18、docs/06:21、docs/06:130、constr/enc_top.xdc:65、constr/enc_top.xdc:274-275、constr/enc_top.xdc:284

1. 烧 ILA 版，抓第一帧 ADC 波形（本次上板的核心验收）

位流（已实测生成，直接用，不要重跑脚本）： **版本号以 `docs/DEV_STATUS.md` §2.x 为权威**（本节此前把版本号写死在这里，已两次过期 ⇒ 改成引用）。 截至 2026-10-11：**生产版 v5 = `_synth_work/enc_top.bit`（§2.6）**、**ILA 版 = `_synth_work/enc_top_ila.bit`**（v4 版为 1,482,840 B @ 2026-10-11 00:39:22 + `.ltx` 67,006 B @ 00:38:31，见 §2 顶部那条；v5 重建后以 §2.6 为准）。 **两版配套的 `.ltx` 必须与 `.bit` 同目录同名同批**（见 §1.2 判据）。

> **★ v5 ILA 版新增了毛刺探针，抓毛刺要靠它**（`docs/DEV_STATUS.md` §1.31 ④）： `dbg_ang_glitch`（单拍脉冲）与 `dbg_ang_delta`（16 位有符号角度差）。 **触发条件请在 Hardware Manager 里手动设成 `dbg_ang_glitch == 1`** —— `tools/ila_debug.tcl` 只建探针与深度，不含触发设置；不设触发就只能靠 20.5 µs 随机窗口 去撞 ~750/s 的毛刺（命中概率 ~0.5%/次）。抓到后请存 CSV 并保留 `.ltx` 的探针顺序。
>
> **★ 这一版（16:12）包含三处修复，必须用这一版**： ① **探针位序修复**（`tools/ila_debug.tcl:126-137`，见 `docs/DEV_STATUS.md` §1.20）—— 08:56 那一版所有多比特探针的值都是**位序反转**的（`dbg_pole_pairs_eff` 显示 8 而配置是 4）； 本版 `enc_top_ila.ltx` 里每条多比特探针的 `subnets` 顺序已经整体翻转，**上板应看到 4。** ✅ **已实测 = 4**（`_synth_work/iladata3.csv`，2026-10-10 17:08，位序判定见 DEV_STATUS §1.23）。 ② **OTR 软清位**（`0x0405[0]`，见 `docs/DEV_STATUS.md` §1.21）—— 写 `0x0405=0x0001` 可清 ADC sticky OTR。 ✅ **已实测生效**；但清位能否"保持 0"取决于输入幅度，见下面 1.9 行（DEV_STATUS §1.23）。 ③ **UART→SPI 桥**（见 `docs/DEV_STATUS.md` §1.22）—— 板载 USB 串口发 4 个字节即可读写任意寄存器（§5.1）。 ✅ **已实测**： `python tools\uart_spi.py --port COM5 read 0x05F0` 回 `5A 18 F0 10 0C AE` ⇒ `DEVICE_ID=0x0C`、 `REV=0x10`（第 2 字节是 `0x053A` 的值，会随状态变，不是固定 `01`）。
>
> ⚠️ **时序口径（2026-10-12 v11 起已收敛，v12 沿用）**： **v11（`RTL_BUILD_ID = 16'h000B`）= v10 相位（两路 `create_generated_clock` 去掉 `-invert`，采样时钟与 `sys_clk` 同相）+ 配套修正的 ADC 输入约束（`-max 14.000 / -min 9.500`）**； **v12（`RTL_BUILD_ID = 16'h000C`，遥测帧布局 v2）沿用同一相位与约束。** 两版位流的 `sys_clk` 域**已全部满足**：生产版 `_synth_work/impl_timing.rpt`（2026-10-12 23:59:16） **WNS +0.959 ns / TNS 0.000 / 0 个失败端点**（11275 端点）、WHS +0.075、WPWS 9.500； ILA 版 `_synth_work/impl_ila_timing.rpt`（2026-10-13 07:11:57）同一结论行 **`All user specified timing constraints are met.`**（WNS +0.332 / 0 失败 / 20165 端点、WHS +0.050）。【v11 历史，已归档 `_synth_work/enc_top_v11prod.bit` / `_synth_work/enc_top_ila_v11.bit`】生产版 2026-10-12 21:15:26 **WNS +0.821 ns / TNS 0.000 / 0 个失败端点**（11310 端点）； ILA 版 21:29:41 同一结论行（布线估计 WNS 0.565 / WHS 0.047）。 **签核一律以生产版为准，ILA 版只作调试用途。** 【历史记录，勿按此复现】v9（反相）相位 + 诚实约束（`-max 24.000 / -min 19.500`）那一版， `impl_ila_timing.rpt` 的 `sys_clk` 域是 **WNS −3.329 ns / TNS −84.745 / 26 个失败端点（全部是 ADC 输入端口）/ 18010 端点**（整体 19161 端点）、结论行 `Timing constraints are not met.` —— 这与生产版 16:05 的 −3.352 是**同一件事**（见 §1.19），不是新增问题；**根因是 ODDR / 生成时钟的 `-invert` 反相相位**（捕获窗口只剩约 4 ns），已在 v11 修复。08:56 那一版的 `+0.527 / 0 失败` 是因为 它构建在诚实约束之前，两者不可比。 【历史记录，同属上面那一版（**非 v11**；v11 两版位流大小见 §2.1）】其余数字：LUT 8822 (42.41%) / FF 8544 (20.54%) / BRAM 7.5 (15.00%) / DSP 17 / IOB 50、 DRC 62 checks 全 `Related violations: <none>`（0 error / 0 critical）、 `Bitstream compression saved 5971264 bits` ⇒ `17537696-5971264 = 11566432 bit = 1,445,804 B`，实测文件 1,445,812 B（同样的 +8 B 头部差）。
>
> **★ 口径说明（2026-10-10 12:08 之后）**： 这两份产物是在 **ADC 输入约束按真实模型重写之前** 构建的（见 `docs/DEV_STATUS.md` §1.19），但 **RTL 与它们是同一版**（RTL 最后改动是 08:37 的 幂等标定修复），**功能与探针集完全一致**，拿上板抓波形没有差别。 重写后只有输入约束变化 ⇒ 生产版重跑那一次

> （12:08）才出现"ADC 输入路径 26 条负余量"的结论。 【历史，v9 反相相位口径】该 26 条 ADC 输入负余量**已由 2026-10-12 的 v11 消除**（约束值 `-max 24.000 / -min 19.500` ⇒ **`-max 14.000 / -min 9.500`**，根因是生成时钟 `-invert` 的反相相位， 不是内部逻辑不收敛）。 ✅ **（2026-10-12 v11 起、v12 再次重跑）已完成重跑**：上板实测 `tPD_meas = 3–4 ns` 合格（第 1.8 步）后，已用当前 XDC **重跑生产版 + ILA 版两版位流**，两版结论行都是 `All user specified timing constraints are met.` （**（2026-10-12 v12）现行：生产版 WNS +0.959 / TNS 0.000 / 0 失败端点 / 11,275 端点，位流 1,042,208 B @2026-10-12 23:59:27；ILA 版 WNS +0.332 / 0 失败 / 20,165 端点，位流 1,428,064 B @2026-10-13 07:12:11；v11 历史为 +0.821 / 11,310 端点、1,086,600 B @21:15:37，已归档 `_synth_work/enc_top_v11prod.bit`）**，"产物时间戳晚于所有约束与 RTL"这条判据重新成立。 原建议文字保留为历史：*"若上板实测 `tPD_meas` 合格（第 1.8 步），建议在最终交付前用当前 XDC 重跑一次两版位流 （`tools\rerun_impl.ps1` 可连跑生产版 + ILA 版），让'产物时间戳晚于所有约束与 RTL'这条判据重新成立。"*

#	操作	判据（数字）	失败回退
1.1	Hardware Manager → Open Target → Auto Connect；Program Device 选 `enc_top_ila.bit`	能连上目标、下载完成	下载卡死/掉线 → **JTAG 频率降到 ≤10 MHz** 再试（见 docs/06:241）
1.2	**同目录加载 `enc_top_ila.ltx`**（必须一起加载；**v12 现行 = 71,337 B @2026-10-13 07:12:00**）	ILA 波形窗里看到 **36 条命名探针**（`u_ila/probe0..35`，共 **269 bit**），不是 `probe0[xx]`（2026-10-10 16:12 那轮曾为 34 条 / 252 bit，属历史）	只有 `probe0[xx]` → `.ltx` 没加载；位流必须与 `.ltx` 同目录同名源（见 docs/05:938、见 docs/DEV_STATUS.md:826-827）
1.3	上电/复位后等 **≥ 21 ms** 再看	所有输出开始活动（POR 自由计数器数满 2^20 拍才释放复位）	一直无反应 → 查 bit 是否真下载（见 docs/06:249）
1.4	示波器探 `J6-26`（`ad_clk`）与 `J6-9`（`ad_clk2`）	都是 **50 MHz**、50% 占空、3.3 V 摆幅，且**两路同相**	无波形 → 查 `J10` 跳线、查 bit；两路反相 → 两路生成时钟约束没生效（见 docs/06:250-251、见 constr/enc_top.xdc:166-173；**v11 起去掉 `-invert`、实测同相**）
1.5	设触发：**`dbg_z_window` 上升沿**（probe33）	抓到非全 0 的一帧（这是"第一次真正抓到波形"的验收条件）	抓不到 → 用 **`dbg_dec_valid`（probe13）或 `dbg_cordic_valid`（probe11）** 作触发；仍不行 → 确认 `C_DATA_DEPTH 1024`、采样时钟是 `u_bufg_clk/O`（见 docs/DEV_STATUS.md:837、见 docs/DEV_STATUS.md:827-828）

#	操作	判据（数字）	失败回退
1.6	看 ADC 原始码：**`dbg_s_raw_11_0_`（probe24）** 与 **`dbg_c_raw_11_0_`（probe7）**	两条都是**二进制补码**（模块输出确是偏移二进制，但 `rtl/ad9238_capture.v:130` `s_calc = ch1_ext - $signed(OFS_CODE)` 已去偏置，`OFS_CODE = 2048`）：**0 V→`000h`、−5 V→`800h`、+5 V→`7FFh`**；10 Vpp 输入时在 **±`7D0h`**（≈±2000 LSB）附近按正弦摆动、**相位差 90°**；不得有大量样本贴死（**贴死 `7FFh` / `800h` 才是削顶**，不是贴死 `000h`）	全 0/全不变 → ① 若输入是 1 Hz：20.5 µs 窗口只占周期 0.002%，**整屏 `0x000` 是正常的**，把输入提到 100 kHz 再看；② 按 1.4 查时钟与跳线；某一位恒定 → 逐位法（见 §5 第 5 条）（见 docs/04:107、见 docs/06:292）
1.7	看 CORDIC 原始角度：**`dbg_angle_raw_15_0_`（probe3）**、环路后 **`dbg_angle_15_0_`（probe2）**	`angle_raw` 在 65536 个码值上单调走完一圈再回绕，**`dbg_dir`（probe14）方向不变**；跨圈看 **`dbg_wrap_pos`（probe31）/ `dbg_wrap_neg`（probe30）** 单拍脉冲	`angle_raw` 出现毛刺/跳变 → 先按 1.8 量 `tPD`；`dbg_dec_valid`（probe13）**每 4 拍**一个 80 ns 脉冲（`LOG2DEC=2`，50 MSPS→12.5 MSPS）；注意 **`dbg_adc_valid`（probe1）恒为 1**，不是每 4 拍（见 rtl/enc_top.v:38、见 docs/06:258）
1.8	**★ 先量 ADC 捕获余量**：示波器双通道同时探 **CH1 = `J6-26`（`ad_clk`，50 MHz）** 与 **CH2 = `J6-38`（`CH1_D11`，符号位）**，量 "`ad_clk` 上升沿 → `CH1_D11` 跳变" 的 `tPD_meas`。**★ 不要探 `D0`**：满幅正弦下 D0 每秒跳变数百万次，5 ns/div 只有 50 ns 屏宽 ⇒ 必然"糊成一片"（不是故障）；`CH1_D11` 只在过零点翻转（1 kHz 输入 ⇒ 20.18 kHz），触发挂 **CH2 上升沿**，5 ns/div、10× / 1 MΩ/DC。记录表见 `docs/09_scope_measure_log.md` §1 测量 A	**（2026-10-12 v11）实测 `tPD_meas = 3–4 ns`**（在 AD9238 手册 `tPD = 2…6 ns` 内）⇒ **判据 `≤4 ns` 通过**，采样沿保持**上升沿**、RTL 不改。由此 v11 建立余量 ≈ **+12.3 ns**（`tPD = 3.5 ns`；`tPD` 2…6 ns ⇒ **+13.8 … +9.8 ns**），与 XDC 相对 `sys_clk` 的推导（**+10.2 … +14.8 ns**）一致。⚠️ **修正旧口径**：**19.5…24 ns 不是手册的 `tPD`**，而是 **v9 反相相位下的总输入延迟预算** = 3.762（源时钟）+ 10.000（半周期，仅反相相位下成立）+ 3.016（ODDR+OBUF）+ `tPD` 2…6 + 1（排线）；旧的"典型件约 +1 ns / 最坏约 −1.6 ns"是那一口径的产物，**v11 已作废**	`tPD_meas ≥ 5 ns` 或 `angle_raw` 有毛刺 ⇒ **按顺序做**（详见 `constr/enc_top.xdc:243-259` 的五步清单）：① 缩短排线（≤5 cm）/ `ad_clk` 串 22~33 Ω；② 最省事的功能级修复 = 把 `rtl/ad9238_capture.v` 的捕获改到 `clk_i` **下降沿**（建立窗口 ~7.8→~17.8 ns，需在捕获后加一级 posedge 重定时）；③ 降采样率（**不能**靠把 `create_clock -period` 改成 40.000，那是物理事实）；④ 最后才 `set_false_path`（必须在 DEV_STATUS 里写明"ADC 输入路径未做时序分析"）。★ **（2026-10-12 v11）实测合格 ⇒ ①②③④ 均不需要执行**（当前约束值见 constr/enc_top.xdc:219-222、v11 改动说明见 constr/enc_top.xdc:215-218）

#	操作	判据（数字）	失败回退
1.9	关掉信号源看 `led[0]` / `led[1]`，`irq_n`	`led[0]` = `AB21` 灭（无超量程）、`led[1]` = `AB22` 灭（幅值正常）；`irq_n`（`K22`）低电平才表示有告警（`otr_lat/amp_low/amp_high/lag_limit/lag_fatal` 之一）	`led[0]` 常亮 → 输入超过 ±5 V 或数据线接错。**若已锁存清不掉**：SPI 写 **`0x0405 = 0x0001`** 清 ADC 内部 sticky OTR（2026-10-10 新增；此前 `clr_otr` 在 `rtl/enc_top.v` 被硬接 `1'b0`，只能重新 Program 器件，按 `KEY0`/降幅度都无效 —— `KEY0` 只做一键标定）。**★ 2026-10-10 上板实测（DEV_STATUS §1.23）：软清位本身一定生效，但"清完几毫秒又亮"= 输入还压在满量程上**（实测 10 Vpp 时 `0x0405` 从 `0x0001` 立即复亮，而把 AFG 幅度降到 **8 Vpp** 后连续 5 轮 ×1 s 全部保持 `0x0000`）。⇒ 先把幅度降到 **≤0.9 满量程（本模块 10 Vpp 满量程 ⇒ 用 8 Vpp）**，再清位；若 8 Vpp 下仍立刻亮，才去量载板 `AD1_OTR`（`J6-39` / `E19`）、`AD2_OTR`（`J6-24` / `B16`）对地电平并查是否悬空。`led[1]` 常亮 → 不是 10 Vpp 或没接信号（见 docs/06:252-253、见 rtl/enc_top.v:814、见 rtl/enc_top.v:818）
1.10	记录并截图本步波形	存：`ad_clk` / `ad_clk2` 同相照、探测器列表 **36 条**（v12/v11；2026-10-10 16:12 轮的 34 条属历史）、`dbg_s_raw` + `dbg_c_raw` 90° 同屏、`dbg_angle_raw` 回绕	见 §3 的照片清单

> ⚠️ 窗口只有 **1024 点 @ 50 MHz = 20.5 µs——1 kHz 信号一个周期 1 ms，一个窗口看不完一个周期**；要看长序列请回到 §2 的 UART 遥测（10 kHz 帧率）。不要写"ILA 看了一个完整周期"（见 docs/DEV_STATUS.md:839-841）。
>
> ⚠️ **两个频率别用错**：本窗口要装下一整圈李萨如图需要 **f_in ≥ 约 48.8 kHz**（20.5 µs 对应 48.8 kHz 一个周期），抓 ADC/角度/相位建议 **100 kHz / 10 Vpp / 0 V 偏置**（窗口内约 2 个周期）；而验 **ABZ 满速插补**要 **f_in ≤ 约 12.2 kHz**（`inter=256` 时标称边沿率 = 4·256·f_in，上限 12.5 MHz = 1 边沿/`filt_valid`，超过就被限速），用 **10 kHz** 最合适。
>
> ⚠️ **位序**：`2026-10-10 08:56` 那版 `enc_top_ila.bit` 的 `.ltx`/CSV 里**所有多比特探针都是位序反转的**（症状：`dbg_pole_pairs_eff` 显示 `8` 而寄存器配置是 `4`、`dbg_stat_byte` 读成 `0x80`）。根因是 `tools/ila_debug.tcl` 把 net 列表按 MSB→LSB 传给 `connect_debug_port`，而该 Tcl 把**列表第一个 net 接成端口 bit0**；已修为 LSB→MSB（`tools/ila_debug.tcl:126-137`）。**重新下载修好的 `enc_top_ila.bit` 后应显示 `4`**；若还在旧位流上看波形/读 CSV，必须对每个多比特列做位序反转才能读（`tools/ila_report.py` 已内置）。

---

2. 烧生产版 + UART 上位机

位流：当前生产版 = `_synth_work/enc_top.bit`，版本号与签核数字以 `docs/DEV_STATUS.md` §2.x 为权威（本节此前写死为 16:05:29 的 804,780 B，已被 §2.4 v3 与 §2.5 v4、§2.6 v5 陆续取代 ⇒ 改成引用，避免"照着

旧数字去烧旧位流"这个已知陷阱）。**烧录前请先核对 `Get-Item _synth_work\enc_top.bit` 的时间戳与 `docs/DEV_STATUS.md` 最新一节的位流时间戳一致。**

> **★ 这一版含 UART→SPI 桥**（`rtl/uart_rx.v` / `rtl/uart_spi_bridge.v`，见 `docs/DEV_STATUS.md` §1.22）： 板载 USB 串口（CH340E， `uart_rx` =T20 / `uart_tx` =W21）发 4 个字节即可读写任意寄存器， 用法见 `docs/06` §5.1 与 `tools/uart_spi.py`。 **★ 盘上还有两份作废的生产版位流**：12:08 的 **1,219,152 B** 与 08:40 的 **962,600 B** （前者是 ADC 输入约束按真实模型重写后的产物，见 `docs/DEV_STATUS.md` §1.19；后者是诚实约束 **之前**的"全约束满足"版）。两者 RTL 都**早于** OTR 软清位与 UART→SPI 桥， **不要再用它们上板**；两份都已另存： `_synth_work/signoff_adc_honest/` 与 `_synth_work/signoff_0840/` （加桥前的整套产物另存 `_synth_work/signoff_pre_uart_bridge/`）。

#	操作 / 命令	判据（数字）	失败回退
2.1	用现成位流：Hardware Manager → Program Device → `_synth_work/enc_top.bit`。若要重跑：`tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build.log` （跑前必须设 `$env:Path`、`$env:HOME` / `$env:USERPROFILE` / `$env:APPDATA`，否则报 `17-1219`）	**判据是产物不是退出码**：日志含 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.`；位流文件**时间戳比所有 `rtl/*.v` 与 `constr/*.xdc` 新**（大小仍以 `docs/DEV_STATUS.md` §2.x 为准； **(2026-10-12 v12) 当前生产版 = `enc_top.bit` 1,042,208 B @2026-10-12 23:59:27、ILA 版 = `enc_top_ila.bit` 1,428,064 B @2026-10-13 07:12:11**；v11 历史 = 1,086,600 B @21:15:37 / 1,458,744 B @21:29:54，已归档 `_synth_work/enc_top_v11prod.bit` / `_synth_work/enc_top_ila_v11.bit`；旧文写死的 804,780 B 是 v5）。**★ v12 已满足"位流时间戳比所有 `rtl/*.v` 与 `constr/*.xdc` 新"**：RTL 最后改动在 2026-10-12，v12 位流时间为 10-12 23:59:27 / 10-13 07:12:11。**★ (2026-10-12 v12) 时序已收敛**：`_synth_work/impl_timing.rpt` @2026-10-12 23:59:16 结论行 **`All user specified timing constraints are met.`**（WNS **+0.959** / TNS 0.000 / **0 失败端点** / **11,275 端点**），ILA 版 `impl_ila_timing.rpt` @2026-10-13 07:11:57 同结论行（WNS +0.332 / 0 失败 / 20,165 端点）； **板上 `0x05F2` 应读回 `0x000C`**（未换版时仍可能是 `0x000B`）； **若仍看到 `Timing constraints are not met.`（26 条 ADC 输入负余量），说明烧的是 v9 或更早的旧位流**（那是历史，见本文 §1 的时序口径）	报 `17-1219` → 补那三行环境变量；产物不对 → 看 `_synth_work/impl_*.rpt`（见 docs/06:146-161、见 docs/DEV_STATUS.md §1.19）
2.1b	用**桥**直接读写寄存器（无需 SPI 主机）： `python tools\uart_spi.py --port COM7 read 0x05F0`	应回 **`5A 01 F0 10 0C B7`**（`0x05F0` DEVICE_ID=0x0C、`0x05F1` REV=0x10）； 需自行 `pip install pyserial`	无输出 → 确认烧的是含桥的位流（2026-10-10 16:05 起； **现行 v12 亦含桥**）； 乱码 → 波特率必须 2000000（见 docs/06 §5.1、见 rtl/SPEC_interface_contract.md §1.12）

#	操作 / 命令	判据（数字）	失败回退
2.2	上位机自检：`python tools\uart_scope.py --selftest`	不接硬件先 PASS（默认波特率参数 `--baud` default = **2000000**）	自检就失败 → 先修 Python 环境，别碰板子（见 tools/uart_scope.py:311、tools/uart_scope.py:313、见 docs/06:450）
2.3	抓数据：`python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --csv adc.csv --png adc.png`	帧率 **10 kHz**（50 MHz / `SAMPLE_DIV=5000`）；`frames > 0` 且 **0 校验错**	`frames = 0` → 按 2.4 顺序查（见 rtl/enc_top.v:44-45、见 rtl/uart_scope.v:132-133、见 docs/06:452）
2.4	收不到/全是坏帧时的**固定顺序**	① **校验和约定**：帧第 17 字节 = `0xAA ^ 0x55 ^`（偏移 2..16 逐字节 XOR）——**含帧头常量**，RTL/上位机/Fixture 三方一致（`sim/fixtures/uart_frame_rtl.hex` 守卫）② **波特率**：必须 2000000（`BAUD_DIV=25` 是精确 2.0000 Mbaud）③ 最后才查 `uwven`/方向位（`0x0109` 的 `[3]abzdir [5]uvwdir`）	① 先核对校验和含不含帧头；② 波特率改 921600 需把 `rtl/enc_top.v` 的 `BAUD_DIV` 由 **25** 改 **54** 并**重出位流**；③ 方向反了写 `0x0109`（见 rtl/uart_scope.v:219-224、见 rtl/reg_file.v:485-489、见 docs/DEV_STATUS.md:579、见 docs/06:457-458）
2.5	看 `s_n`/`c_n`（帧偏移 4/6，Q1.15 有符号）	两条**相位差 90°** 的正弦；幅度接近 **±32767 的 60%~100%**（≥19660）；两条都不得是常数	两条都是常数 → 回 1.4（`ad_clk` 没到载板）；幅度只有几百 → 信号源不是 10 Vpp（见 docs/06:319-324、见 rtl/uart_scope.v:14-15）
2.6	看 `angle`（偏移 2）：一个电周期内**单调 0 → 65535 再回 0**；`pos_full`（偏移 12）按每圈 `POLE_BITS` 个电周期阶梯上升	单调无回跳、无 ±180° 折叠	`angle` 在 ±180° 附近跳变 → 象限折叠，重跑 `tb_enc_top`（本版已覆盖 262140 点）（见 docs/06:345-347）

#	操作 / 命令	判据（数字）	失败回退
2.7	看 `stat`（偏移 16）	**静止/匀速时 `stat = 0x00`**；只有 `bit3 inclim` 可能因 `inter=256/div=1` 高细分而置起（对应 `led[2]` 亮，**属预期**）。★ **`bit0 oflow` 长期为 1 通常不是活故障**：`oflow = cor_sat | otr_lat`，而模块的 OTR 引脚会被自身毛刺反复拉高并**sticky 锁存**（§1.31 ①）——`tools/uart_scope.py` / `tools/calib_repeat.py` 已在采数前自动写 `0x0405=0x00FF` 清一次	`bit0 oflow` **清位后几秒内又复亮** ⇒ 就是模块 OTR（正常现象，见 §1.31 ①）；若 `0x053A[1] amplo/[2] amphi` 也置起 ⇒ 才是真超量程。`bit5 scamp` → 幅值越限；`bit4 lagfatl` → 致命滞后（见 docs/06:400-412、见 docs/06:428-429）
2.7b	**看 ABZ/UVW 电平 + 告警位（v12 新增，不用 ILA）**：**推荐一条命令** `python tools\demo_abz_uvw.py --port COM3 --seconds 30 --archive`（自带版本门禁 `0x05F2 = 0x000C`、寄存器自配、每秒滚动统计、9 项逐项判定、归档 CSV+PNG，末行打印 `DEMO PASS` / `DEMO FAIL(n)`；`--selftest` 可离线验证判定逻辑）；手工方式 `python tools\uart_scope.py --port COM3 --baud 2000000 --seconds 30 --csv io.csv --png io.png`（端口号随插拔变，用 `python tools\uart_spi.py ports` 查）	帧布局 v2 判据：**偏移 9 = `0xA2`**（读到别的值 ⇒ 烧的是 v11 或更早的旧位流，帧里没有电平）；偏移 8 = `io_state`（bit0..5 `a b z u v w`、bit6 `dir`、bit7 `limiter`）；偏移 10 = `io_flags`。CSV 多出 `layout/io_state/io_flags/a..w/dir/limiter` 列；PNG 第 3 行是**等效采样**（按电角度分箱重画）⇒ A/B 正交、UVW 六步 `001→011→010→110→100→101`、Z 每电周期一个窄脉冲。★ **已实测（2026-10-13 12:29 起，`0x05F2 = 0x000C`）**：30 s / 298,749 帧 / `inter` = 32 时 A 50.0% / B 49.9% / Z 0.1%、UVW 六态全覆盖且**等效采样 64/64 箱**，`DEMO PASS`，归档 `deliverables/figures/telemetry_abz_uvw.png`（120,686 B / `CF8D3B1985B06703…`）+ `deliverables/data/telemetry_abz_uvw.csv`（25,237,925 B / `1EEC84B077AE8EAA…`，已进 `SHA256SUMS.txt`）；上电默认 `inter` = 256 时 `lag_warn/lag_fatal` 常置 1（已知限频现象，拍前先写 `0x0101 = 0x0020`）	六路全低 ⇒ 先写 `0x0109=0xC0`（开 `abzen` / `uwven`）；按时间轴看 A/B"糊成一团" ⇒ **正常**（边沿率可到 MHz ≫ 10 kHz 帧率），看第 3 行等效采样图或上示波器；UVW 出现 `000` / `111` ⇒ 报告会直接告警；偶见极少数帧被判 v1（占比 < 0.1%）⇒ 是 XOR 单字节校验漏网的多比特误码，**不是**旧位流（见 docs/06 §4.2、见 rtl/uart_scope.v:192-266、见 tools/uart_scope.py、见 tools/demo_abz_uvw.py、见 docs/DEV_STATUS.md §1.59）
2.8	一键标定：按板上 **`KEY0`（`key_n` = `W9`）**，或 SPI 写 `0x0502 = 0x0001`	`led[3]`（`J5-19`）亮一下表示正在标定；标定后 `--png` 的李萨如图更接近正圆	标定后更差 → 先确认激励是 1 Hz 慢扫、无堵转（见 docs/06:379-383、见 docs/06:486、见 rtl/enc_top.v:760）

#	操作 / 命令	判据（数字）	失败回退
2.9	UVW（可选）：SPI 写 `0x0109 = 0xC0`（保留 `abzen`、打开 `uwven`）	`u/v/w` 三相互差 **120°**、各 **50%** 占空；此时若要精确验 A 边沿数，先写 `0x0102 = 0x1C`（`div=8`，A 边沿 = `4·256/8 = 128` 个/电周期 = 64 Hz）	`u/v/w` 全低 → **上电默认 `uwven=0`**，`rtl/uvw_gen.v:597-599` 三相强置 0，必须 SPI 打开（见 docs/06:376-377、见 docs/06:388-392、见 docs/DEV_STATUS.md:481）
2.10	**v6 起可查的毛刺诊断**（地址表见 `docs/DEV_STATUS.md` §1.32 ④）：推荐一键 `python tools\diag_counters.py --port COM6 --seconds 20`（自动两次求差、自带 `--selftest`）；手工则两次执行 `python tools\uart_spi.py --port COM6 read 0x0505 0x0507 0x0509 0x050B 0x050D 0x0540`，用**两次的差**相除	`0x05F2` **必须 ≥ 6**（v6 = 6 / v7 = 7 / v8 = 8 / **v9 = 9，含 `GATE_STATE=1` + kp2/ki3 + Z 脉宽修复（截断）+ Z 宽度 +1（手册口径）/ v11 = 11 / v12 = 12（现行，板上 `0x05F2` 应读回 `0x000C`）**；读到 0 ⇒ 是 v5 或更早）；毛刺密度 = Δ毛刺计数 ÷ Δ `0x0540` 周期脉冲 = **每输入周期毛刺次数**；若 `Δ>5.6°` 与 `Δ保持动作` 相等 ⇒ 每一拍都被 5b 保持挡住、无连续 2 拍；`int_max/int_min > 1.25` ⇒ 该窗口按 §1.29 判非整周期并作废	计数器**只在配置/POR 时清零**（读的是自配置起累计）；窗口快照只在**标定关门**时更新（未标定时两次读数必然相同，不是卡死）；`0x05F2 = 0` ⇒ 位流是 v5 或更早；`> 11.25°` 一档（`0x0507/08`）恒 0 ⇒ 角度跳变根本没到能翻滞回的幅度，旗标要从别处找（见 §1.32 ⑥）
2.11	**v6 实测基线**（2026-10-11 13:2x，用于以后复测对照）：`python tools\calib_repeat.py --port COM6 --n 20` + `python tools\diag_counters.py --port COM6 --seconds 20`	20/20 GOOD；`retry {0:20}`；`win_bad 0/20`；`OFS_S` 53…54、`OFS_C` 352…359、`GAIN_S` 33060…33065、`GAIN_C` 33347…33354、`PHASE_K` 97；毛刺密度 **0.00444 次/周期**（`Δ>11.25° = 0`）；窗口 `int 12498/12506` 比值 **1.0006**；累计分档 486/8/8/3	明显偏离上面数字才需查：`retry` / `win_bad` 非 0 ⇒ 有连续 2 拍毛刺漏进 `angle_filt`；`> 11.25°` 非 0 ⇒ 要按 §1.32 ③ 的分工继续分档分析；`trunc = 1` 每轮都属**设计行为**（§1.32 ⑦），不是故障

默认配置（先记住，不然会误判）：`0x0100=0x04`（极对数 4）、`0x0101/02=0x00/0x00`（`inter=256`、`div=1`）、`0x0108=0x04`（`zwidth=4`，**上电即有 Z**；**★ v9 起脉冲宽 = `zwidth+1` = 5 个 AB 边沿步**，1 kHz/inter=256/div=1 时 ≈4.88 µs；v8 及以前为 4 步 ≈3.9 µs，见 `docs/DEV_STATUS.md` §1.49）、`0x0104=0x00`（回退编译期档 `UVW_HYST_SHIFT=6`）、`0x0109=0x40`（`abzen=1`、**`uwven=0`**）、`0x0106=0x00`（限频最松）（见 docs/06:275-280、见 rtl/reg_file.v:226-240）。

SPI 包格式：**模式 0（CPOL=0/CPHA=0）、MSB 先、32 位、≤10 MHz**，`bit31 st=1`、`bit30:29 rm=00`、`bit28:27 wm=01`、`bit26:24 page`、`bit23:16 addr[7:0]`、`bit15:0 data`（见 docs/06:359-369）。

---

3. 输出与信号完整性（示波器测点与判据）

#	测点	判据（数字）	出处
3.1	`J5-5`（`a_out` / `H20`）、`J5-6`（`b_out` / `G20`）	A/B 正交方波；`div=8` 时 A 边沿 **64 Hz**（0.5 Hz 输入）；`inter=256/div=1` 下 1 电周期 = `4·inter/div = 1024` 个 A 边沿	见 docs/06:285、docs/06:335、docs/06:388
3.2	`J5-7`（`z_out` / `M21`）	**每 64 个输入周期 1 个 Z**（= f_in/64；1 kHz ⇒ 15.625 Hz / 64 ms）；宽度 = `0x0108` 个 **AB 边沿步**（默认 4；1 kHz/inter=256/div=1 时 1 步 = 977 ns ⇒ 3.9 µs），看不清就写 `0x0108 = 0x000F`（15 步 ⇒ 14.6 µs）	见 DEV_STATUS §1.26、docs/06:391-475
3.3	`J5-8/9/10`（`u/v/w` = `L21` / `N22` / `M22`）	写 `0x0109=0xC0` 后：三相互差 **120°**、各 **50%** 占空	见 docs/06:392、docs/06:476
3.5	边沿质量	排线过长/无串阻会振铃：**缩短线长（≤5 cm）或在时钟线串 22~33 Ω**	见 docs/04:286
3.6	示波器照片清单（答辩与报告用）	① `ad_clk` + `ad_clk2` 同相（1.4）② `ad_clk` 上升沿 → `CH1_D11`（`J6-38`）`tPD_meas`（1.8；★ **不要用** `D0`：满幅正弦下会糊成一片）③ `J5-5/6/7` A/B/Z 同屏（3.1/3.2）④ `J5-8/9/10` 三相互差 120°（3.3）⑤ UART 李萨如图 `--png`（2.8）	见 docs/06:470-477。★ **逐条操作与记录表见 `docs/09_scope_measure_log.md`**（tPD / 温度与器件批次复测 / Z 脉宽带游标截图，2026-10-12 起用）

4. 已知坑与规避（逐条）

#	坑	规避 / 判据	出处
4.1	Vivado 脚本在 **`Bitgen Completed Successfully` 之后会挂住**	`tools/impl_check.tcl` 与 `tools/ila_debug.tcl` 都会（`PROD_EXIT=-1` / `ILA_EXIT=-1`）。**判据永远是产物本身**（时间戳 + 报告 + 位流大小/sync 字），**不是脚本退出码**	见 docs/DEV_STATUS.md:894-897、docs/DEV_STATUS.md:949、docs/DEV_STATUS.md:1104-1105

#	坑	规避 / 判据	出处
4.2	ILA 需要 **GOLD/ChipscopePro** 许可，BASIC 会报 `ERROR: [Vivado 12-29205] ... Your current selected license is BASIC`	先设 `$env:XILINXD_LICENSE_FILE = 'D:\Vivado\Xlnx_2026_1.lic'`，启动日志要出现 `A valid Vivado Design Suite GOLD license has been detected.`	见 docs/06:167、docs/06:171-184、docs/06:501
4.3	生成 ILA 的脚本会**派生带管道的子 vivado 进程**，受限沙箱下父进程无限挂死	只对该脚本单独放宽一次权限；生产版 `tools/impl_check.tcl` 不生成 IP、不需要	见 docs/DEV_STATUS.md:1103、docs/05:904
4.4	`uwven` 上电为 0，UVW 恒低	必须 SPI 写 **`0x0109 = 0xC0`**；这不是坏了	见 docs/06:296、docs/DEV_STATUS.md:477-481
4.5	`zwidth` 不能是 0	0 会让 `z_pulse_n` 恒 0，Z 与 UVW 一起被钉死；复位默认 `0x0108 = 0x04`	见 docs/DEV_STATUS.md:483-484、rtl/reg_file.v:239
4.6	UART 校验和**含帧头常量**（`0xAA ^ 0x55 = 0xFF`）	定案取 RTL 一方（改主机端与文档，不改 RTL）：`chk = payload_xor ^ 0xFF`。**改动前先看 `sim/fixtures/uart_frame_rtl.hex` + `tools/check_uart_frame_fixture.py`**，不要动这条约定	见 docs/DEV_STATUS.md:569-580、rtl/uart_scope.v:219-226（v12 帧布局 v2 同样把 `io_state ^ 0xA2 ^ io_flags ^ 0x00` 异或进去，别只异或载荷）

#	坑	规避 / 判据	出处
4.7	`J9` / `J10` 出厂**不保证 3.3 V**，由 `U22` 经跳线选压	上电前拨到 3.3 V；报 VCCO 不符就是这条	见 docs/04:16、docs/04:256-259、docs/06:511
4.8	遥测链路**占用率 90.7%、余量 9.3%**	一帧 `17*(10*25+2)+10*25 = 4534 clk = 90 µs`，`latch` 节拍 `5000 clk = 100 µs`；想把 `SAMPLE_DIV` 调小或 `BAUD_DIV` 调大之前，先按这个算式复核	见 docs/DEV_STATUS.md:485-490
4.9	ABZ/UVW 六个输出**没有端接/串阻**（约束与驱动强度已补，硬件侧没做）	上板若看到 ABZ 偶发多计/漏计，**第一个怀疑这里**	见 docs/DEV_STATUS.md:505-510
4.10	XDC 里曾有**自相矛盾**的注释（**2026-10-10 已修**）	旧文第 130 行说 "`ad_clk` 比 `clk50` 晚 10 ns"，旧第 162-165 行却说两者在引脚上"同一物理时刻"——后者已被证伪。现 `constr/enc_top.xdc:189-211` 保留该断言的**历史记录并逐条给出反证。★ (2026-10-12 v11)**：两路 `create_generated_clock` 去掉 `-invert`（相位对照证据见 `constr/enc_top.xdc:158-165`，v11 约束见 `:166-173`），ADC 输入约束由 `max 24.000 / min 19.500` 改为 **`max 14.000 / min 9.500`**（现值为 `:219-222`，改动说明在 `:215-218`；**旧的 24.000/19.500 是 v9 反相相位下的诚实值，作为历史保留在 `:207-209`**），**ADC 输入路径负余量随之消除**（两版位流均 `All user specified timing constraints are met.`）。五步处理清单仍在 `:243-259`（v11 实测 `tPD_meas = 3–4 ns` 后确认**不必执行**）。看到别处还写"3 ns 余量"或把 `24.000/19.500` 当现值即为过期说法	见 constr/enc_top.xdc:189-211、constr/enc_top.xdc:215-218、constr/enc_top.xdc:219-222、constr/enc_top.xdc:243-259
4.11	ILA 观察窗口 = 1024 点 = **20.5 µs**	1 kHz 一个周期 1 ms，窗口看不完一个周期；长序列用 UART 遥测	见 docs/DEV_STATUS.md:839-841
4.12	旧位流坑	`enc_top_pre_p02.bit`（726,924 B @ 2026-10-10 02:32:17）**早于** P0-2 输出约束与 §1.18 幂等标定修复，不要拿到板上用；RTL 之后任何改动都必须重出位流	见 docs/DEV_STATUS.md:998-1001、docs/DEV_STATUS.md:471-472
4.13	**UART→SPI 桥**（2026-10-10 新增，`rtl/uart_rx.v` + `rtl/uart_spi_bridge.v`）	① 只有**含桥的新位流**才有这个功能（桥之前的位流发 4 字节没有任何反应）；② 波特率固定 **2 Mbaud**（与遥测同一 `BAUD_DIV=25`），发完命令**必须等 6 字节应答再发下一条**（桥一次只处理一条命令，首字节前还要求线路空闲 ≥13 位）；③ 写命令的应答是**上一帧**的内容、没有语义，只表示"链路通、已执行"；④ 遥测（`AA 55 …` 18 字节帧）与应答（`5A …` 6 字节）共用 TX，`tools/uart_spi.py` 会自动跳过遥测帧；⑤ 外部 SPI 主机拉低 `spi_ss_n` 期间桥不发帧，等 2 ms 就放弃该命令（不写、不应答），重发即可；⑥ 需要 `pyserial`（`pip install pyserial`），`--selftest` 与 `--file` 不需要	见 rtl/SPEC_interface_contract.md §1.12、见 docs/06:396-435、见 sim/logs/tb_uart_spi_bridge.log

#	坑	规避 / 判据	出处
4.14	板载 USB 串口是 **CH340E**（不是 FT2232）	`uart_rxd` = **T20**（CH340E 的 TXD → FPGA，桥的输入）、`uart_txd` = **W21**；`docs/04:222` 写 FT2232 是错的（那是 JTAG 那一路）。USB Type-C 口同时给整板 5 V 供电，断电能仍识别 COM 号	见 docs/ATK_board_digest.md:61、见 docs/ATK_board_digest.md:221-222、见 constr/enc_top.xdc:303

5. 速查表

> ★ **已提到文首**（见上面的「★ 一页速查表」）——上板当天只需要看那一张表 + 本文件第 4 节（已知坑）。 内容不再在此重复，避免两处表格分叉；要改速查表请改文首那一份。

6. 时间预算与 R2/G1 回退

先说清楚：`docs/06` 里**没有任何时间预算/分钟级阈值**（我已 grep 过全文，"30 分钟"这类数字不存在）。下面只写 `docs/00_roadmap_30d.md` 里已有的回退路径。

触发条件	出处	立即执行（按顺序）
"若 30 分钟内没抓到 ADC 波形"（本次上板自定的目标，**路线图里没有这个数字**）	本地约定，无文件出处	按下面 G1 的三级降级走
G1 决策门：ILA 里能看到 12 位 ADC 数据随输入正弦变化	见 docs/00:67	**降采样率到 25 MHz → 换相位 0/90/270 → 最后退到正点原子 8bit AD9280 模块（有官方例程） ★ 2026-10-15 标注：G1 已通过，且"降采样率到 25 MHz"这一档已作废 —— 见本表末行"ADC 输入路径负余量"**
R2 风险：180° 捕获相位不对，数据全错（触发信号：ILA 数据全是 0 或随机；影响：阻塞 3 天）	见 docs/00:246	**扫 0/90/180/270 → ~~降到 25 MHz~~（★ 2026-10-15 标注：已作废，见本表末行"ADC 输入路径负余量"）→ 必要时上 IDELAY**
R3 风险：AD9238 速度等级不是 -65	见 docs/00:247	**先跑 25~40 MHz；看丝印确认等级**

触发条件	出处	立即执行（按顺序）
ADC 输入路径负余量（**v9 反相相位下的历史问题；2026-10-12 v11 已消除**，根因 = ODDR / 生成时钟 `-invert`）	见 constr/enc_top.xdc:243-259（历史处置清单）、constr/enc_top.xdc:215-218（v11 改动）	**（2026-10-12 v11 起、v12 沿用）已完成：实测 `tPD_meas = 3–4 ns`（① ✅）、约束改为 `-max 14.000 / -min 9.500`、两版位流 `All user specified timing constraints are met.`**（v12 生产版 WNS +0.959 / TNS 0.000 / 0 失败端点 / 11,275 端点；v11 历史为 +0.821 / 11,310 端点）⇒ **② 缩短排线/端接、③ 降 25 MHz、④ `set_false_path` 全部不需要；清单保留仅作历史与旧位流排障**
上位机来不及	见 docs/00:250	**退到 ILA + 串口十六进制打印**
整体降级原则	见 docs/00:61-63、docs/00:74	交上去的必须是"能跑、能演、数据完整"的系统；**过不了决策门就立刻降级，不要硬扛**

附：我核到的不一致（引用时请注意）

1. `docs/DEV_STATUS.md:289` 说"本机没有 ILA 可用（BASIC 许可证）"，与 `docs/06:26-27`、`docs/DEV_STATUS.md:543` 的"许可证已升级到 GOLD，ILA 现已可用"矛盾——**以 GOLD 可用为准**（本文按 GOLD 写）。
2. `docs/06:28-29` 说内部信号观测"全部走 UART 遥测"，但 `docs/06:163` 起又给出完整的 ILA 流程（本节 1 就是按 ILA 写的）——两者口径不一致，实测按"ILA 抓边沿/首帧、UART 抓长序列"用。
3. 路线图 `docs/00:92` 写"数据在**下降沿**捕获"，而 `constr/enc_top.xdc:143` 与 `rtl/enc_top.v:129` 都是 **clk50 上升沿**捕获（**2026-10-12 v11 起 `ad_clk` 与 `sys_clk` 同相**，不再是 v9 的"反相 180°"；见 `constr/enc_top.xdc:166-173`）——以 XDC/RTL 为准。
4. `docs/06` 全文没有时间预算/分钟阈值，第 6 节只能引用路线图的 G1/R2/R3 回退路径。